LAPORAN PRAKTIKUM

POSTTEST 6

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Daniel Vivaldi Sagala (2609106054)

Kelas (B2 ‘26)

PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

2026

1. Flowchart

(Pada bagian ini, Anda diminta untuk melampirkan flowchart beserta penjelasan singkat yang menggambarkan alur logika program, mulai dari input, proses, hingga output. Pastikan flowchart dibuat dengan simbol yang sesuai untuk mempermudah pemahaman.)

Catatan:

- Jika flowchart terlalu panjang + ribet pakai **on-page atau off-page reference** untuk memecah fiturnya biar ga burem karena kepanjangan(khusus drawio)

2. Deskripsi Singkat Program

Sistem Pendataan Kendaraan adalah aplikasi berbasis konsol yang digunakan untuk mengelola data kendaraan, seperti nomor plat, nama pemilik, jenis kendaraan, merek, dan tahun kendaraan. Program ini dilengkapi dengan fitur login dan registrasi akun yang memiliki dua tingkat akses, yaitu Admin dan User. Pengguna dapat menambahkan, melihat, mengubah, dan menghapus data kendaraan sesuai dengan hak aksesnya, sedangkan Admin memiliki hak akses tambahan untuk melihat, mengedit, dan menghapus akun pengguna. Program ini juga menerapkan validasi input, pembatasan percobaan login sebanyak tiga kali, serta konfirmasi sebelum penghapusan data. Program ini bertujuan untuk mempermudah pengelolaan data kendaraan sekaligus menerapkan konsep dasar pemrograman Python, seperti percabangan, perulangan, dictionary, validasi data, autentikasi, dan operasi CRUD (Create, Read, Update, Delete).

3. Source Code

A. Fitur Menampilkan Menu Utama

Menampilkan tiga pilihan utama, yaitu Login, Register, dan Keluar dari program.

Source Code:

```
while True:
    os.system("cls" if os.name == "nt" else "clear")
    print("🏍️ SISTEM PENDATAAN KENDARAAN 🚗")
    print("1. Login")
    print("2. Register")
    print("3. Keluar")

    opsi = input("Pilih Menu : ")
```

B. Fitur Login

1. Pengguna memasukkan username dan password.
2. Sistem mencocokkan data dengan daftar akun yang tersimpan.
3. Terdapat batas maksimal tiga kali percobaan login.
4. Jika login berhasil, sistem menampilkan username dan level akses pengguna.
5. Jika login gagal tiga kali, pengguna harus memulai ulang program untuk mencoba kembali.

Source Code:

```
percobaan = 0
login_berhasil = False

        while percobaan < 3 and not login_berhasil:
            username = input("Masukkan username: ").strip()
            password = input("Masukkan password: ").strip()

            role = ""
            if (
                username in akun
                and akun[username]["password"] == password
            ):
                login_berhasil = True
                role = akun[username]["role"]
            else:
                percobaan += 1
                sisa = 3 - percobaan
                if sisa > 0:
                    print(f"Username atau password salah! Sisa percobaan:
{sisa}")
                else:
                    print("Username atau password salah 3 kali. Silahkan mulai
ulang")
                    input("\nTekan Enter untuk melanjutkan...")

        if login_berhasil:
            print("Login berhasil!")
            print("Selamat datang,", username)
            print("Role:", role)
```

C. Fitur Register

1. Pengguna baru dapat membuat akun dengan memasukkan username dan password.
2. Sistem memeriksa apakah username sudah digunakan.
3. Jika username belum terdaftar dan data tidak kosong, akun baru akan disimpan.
4. Setiap akun yang didaftarkan secara otomatis memiliki level akses User.

Source Code:

```
username_baru = input("Buat username: ").strip()
password_baru = input("Buat password: ").strip()

username_ada = False

for user in akun:
    if user == username_baru:
        username_ada = True

if username_baru == "" or password_baru == "":
    print("Error: Username dan password wajib diisi.")
elif username_ada:
    print("Error: Username sudah digunakan.")
else:
    akun[username_baru] = {
        "password": password_baru,
        "role": "User"
    }

    print("Registrasi berhasil!")
    print("Silakan login menggunakan akun baru.")
    input("\nTekan Enter untuk melanjutkan...")
```

D. Fitur Hak Akses Berdasarkan Level

Program memiliki dua level akses, yaitu Admin dan User.

1. Admin: dapat mengelola data kendaraan melalui fitur tambah, lihat, ubah, dan hapus kendaraan. Admin juga dapat melihat, mengedit, dan menghapus akun pengguna dengan ketentuan tertentu.
2. User: dapat menambahkan, melihat, mengubah, dan menghapus data kendaraan miliknya sendiri. User tidak memiliki akses ke menu pengelolaan akun pengguna.

Source Code:

```
print("1. Tambah Kendaraan")
print("2. Lihat Kendaraan")
print("3. Ubah Kendaraan")
print("4. Hapus Kendaraan")

if role == "Admin":
    print("5. Lihat Data Pengguna")
    print("6. Edit Data Pengguna")
    print("7. Hapus Akun Pengguna")

print("0. Logout")
```

E. Fitur Menu Pendataan Kendaraan (CRUD)

1. Tambah Kendaraan (Create)

Pengguna dapat menambahkan data kendaraan baru. Sistem memvalidasi nomor plat, nama pemilik, jenis kendaraan, merek, dan tahun kendaraan sebelum menyimpan data.

Source Code:

```
plat = input("Nomor plat: ").strip().upper()

plat_ada = False

for data in data_kendaraan.values():
    if data["plat"] == plat:
        plat_ada = True
        break

if plat == "":
    print("Error: Nomor plat wajib diisi.")
elif plat_ada:
    print("Error: Nomor plat sudah terdaftar.")
else:
    pemilik = input("Nama pemilik: ").strip()
    jenis = input(
        "Jenis kendaraan (Mobil/Motor): "
    ).strip().capitalize()
    merek = input("Merek kendaraan: ").strip()
    tahun = input("Tahun kendaraan: ").strip()

    tahun_valid = (
        tahun.isdigit()
        and len(tahun) == 4
        and 1899 <= int(tahun) <= 2676
    )

    if pemilik == "" or merek == "":
        print("Error: Data tidak boleh kosong.")
    elif jenis not in ["Mobil", "Motor"]:
        print("Error: Jenis harus Mobil atau Motor.")
    elif not tahun_valid:
        print("Error: Tahun kendaraan tidak valid.")
    else:
        data_kendaraan[id_berikutnya] = {
            "plat": plat,
            "pemilik": pemilik,
```

```
            "jenis": jenis,
            "merek": merek,
            "tahun": int(tahun),
            "username": username
        }

        print("Data kendaraan berhasil ditambahkan!")
        print("ID kendaraan:", id_berikutnya)
        id_berikutnya += 1
```

2. Lihat Kendaraan (Read)

Menampilkan seluruh data kendaraan yang tersimpan, meliputi ID, nomor plat, nama pemilik, jenis, merek, tahun, dan pengguna yang memasukkan data.

Source Code:

```
if len(data_kendaraan) == 0:
    print("Belum ada data kendaraan.")
else:
    for id_kendaraan, data in data_kendaraan.items():
        print("-----------------------------------")
        print("ID           :", id_kendaraan)
        print("Nomor Plat   :", data["plat"])
        print("Nama Pemilik :", data["pemilik"])
        print("Jenis        :", data["jenis"])
        print("Merek        :", data["merek"])
        print("Tahun        :", data["tahun"])
        print("Input Oleh   :", data["username"])
```

3. Ubah Kendaraan (Update)

Pengguna memasukkan ID kendaraan yang ingin diubah. Sistem memeriksa keberadaan data dan hak akses sebelum memperbarui informasi kendaraan.

Source Code:

```
id_input = input("Masukkan ID kendaraan: ").strip()

if not id_input.isdigit():
    print("Error: ID harus berupa angka.")
else:
    id_kendaraan = int(id_input)

    if id_kendaraan not in data_kendaraan:
        print(
            "Error: Data tidak ditemukan "
            "atau Anda tidak memiliki akses."
        )
    else:
        data = data_kendaraan[id_kendaraan]

        if role != "Admin" and data["username"] != username:
            print(
                "Error: Data tidak ditemukan "
                "atau Anda tidak memiliki akses."
            )
        else:
            # Data nya divalidasi dulu
            data["plat"] = plat_baru
            data["pemilik"] = pemilik_baru
            data["jenis"] = jenis_baru
            data["merek"] = merek_baru
            data["tahun"] = int(tahun_baru)

            print("Data berhasil diperbarui!")
```

4. Hapus Kendaraan (Delete)

Pengguna memasukkan ID kendaraan yang ingin dihapus. Sistem memeriksa hak akses dan meminta konfirmasi sebelum menghapus data.

Source Code:

```
id_input = input("Masukkan ID kendaraan: ").strip()

if not id_input.isdigit():
    print("Error: ID harus berupa angka.")
else:
    id_kendaraan = int(id_input)

    if id_kendaraan not in data_kendaraan:
        print(
            "Error: Data tidak ditemukan "
            "atau Anda tidak memiliki akses."
        )
    else:
        kendaraan = data_kendaraan[id_kendaraan]

        if role != "Admin" and kendaraan["username"] != username:
            print(
                "Error: Data tidak ditemukan "
                "atau Anda tidak memiliki akses."
            )
        else:
            print("Data yang akan dihapus:")
            print("ID           :", id_kendaraan)
            print("Nomor Plat   :", kendaraan["plat"])
            print("Nama Pemilik :", kendaraan["pemilik"])
            print("Jenis        :", kendaraan["jenis"])
            print("Merek        :", kendaraan["merek"])
            print("Tahun        :", kendaraan["tahun"])

            konfirmasi = input(
                "Yakin hapus data? (y/n): "
            ).strip().lower()

            if konfirmasi == "y":
                data_kendaraan.pop(id_kendaraan)
                print("Data berhasil dihapus!")
            elif konfirmasi == "n":
                print("Penghapusan dibatalkan.")
            else:
                print("Error: Pilihan harus y atau n.")
```

F. Fitur Pengelolaan Akun oleh Admin

Fitur ini hanya tersedia untuk Admin dan mencakup tiga fungsi berikut.

1. Lihat Data Pengguna

Menampilkan daftar akun yang terdaftar, termasuk nomor urut, username, dan level akses.

Source Code:

```
if len(akun) == 0:
    print("Belum ada data pengguna.")
else:
    i = 1

    for username_akun, data in akun.items():
        print("--------------------------------")
        print("Nomor   :", i)
        print("Username:", username_akun)
        print("Role    :", data["role"])
        i += 1
```

2. Edit Data Pengguna

Admin dapat mengubah username, password, dan level akses pengguna. Sistem memvalidasi username agar tidak duplikat serta membatasi pilihan role menjadi User atau Admin.

Source Code:

```
if username_baru == "" or password_baru == "":
    print("Error: Username dan password wajib diisi.")
elif username_ada:
    print("Error: Username sudah digunakan.")
elif role_baru not in ["User", "Admin"]:
    print("Error: Role harus User atau Admin.")
else:
    akun[username_baru] = akun.pop(username_lama)
    akun[username_baru]["password"] = password_baru
    akun[username_baru]["role"] = role_baru

    for kendaraan in data_kendaraan.values():
        if kendaraan["username"] == username_lama:
            kendaraan["username"] = username_baru

    print("Data pengguna berhasil diperbarui!")
```

3. Hapus Akun Pengguna

Admin dapat menghapus akun pengguna dengan konfirmasi. Sistem mencegah penghapusan akun sendiri dan akun yang memiliki role Admin.

Source Code:

```
if username_target == username:
    print("Error: Tidak dapat menghapus akun sendiri.")
elif akun[username_target]["role"] == "Admin":
    print("Error: Akun Admin tidak boleh dihapus.")
else:
    konfirmasi = input(
        "Yakin hapus akun? (y/n): "
    ).strip().lower()

    if konfirmasi == "y":
        akun.pop(username_target)
        print("Akun pengguna berhasil dihapus!")
    elif konfirmasi == "n":
        print("Penghapusan dibatalkan.")
    else:
        print("Error: Pilihan harus y atau n.")
```

G. Fitur Logout dan Keluar Program

Fitur logout mengakhiri sesi pengguna dan mengembalikan program ke menu utama. Sementara itu, menu keluar menghentikan program.

Source Code:

```
# Logout
elif menu == "0":
    masuk_menu = False
    os.system("cls" if os.name == "nt" else "clear")
    print("Anda berhasil logout.")

# Keluar program
elif opsi == "3":
    print("Terima kasih telah menggunakan Program ini!")
    break
```

4. Hasil Output

```
========== REGISTER ==========
Buat username: Rifqi
Buat password: 7676
Registrasi berhasil!
Silakan login menggunakan akun baru.

Tekan Enter untuk melanjutkan...
```

Gambar 4.1 Register

```
=== LOGIN ===
Masukkan username:
Masukkan password:
Username atau password salah! Sisa percobaan: 2
Masukkan username:
Masukkan password:
Username atau password salah! Sisa percobaan: 1
Masukkan username:
Masukkan password:
Username atau password salah 3 kali. Silahkan mulai ulang

Tekan Enter untuk melanjutkan...
```

Gambar 4.2 Login

```
SISTEM PENDATAAN KENDARAAN .
1. Login
2. Register
3. Keluar
Pilih Menu : 3
Terima kasih telah menggunakan Program ini!
vario@vario:~/Downloads/Tugas Praktikum/Tugas APD/posttest/posttest-apd-5
$
```

Gambar 4.3 Keluar

```
========= HAPUS KENDARAAN =========
Masukkan ID kendaraan: 1
Data yang akan dihapus:
[1, 'KT 1234 AB', 'Robin', 'Mobil', 'Toyota Avanza', 2022, 'Daniel']
Yakin hapus data? (y/n): y
Data berhasil dihapus!

Tekan Enter untuk melanjutkan...
```

Gambar 4.4 Delete

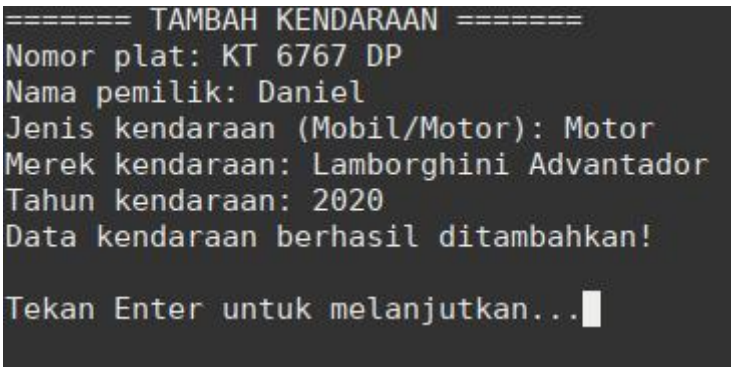
```
========= UBAH KENDARAAN =========
Masukkan ID kendaraan: 2
Data lama: [2, 'KT 5678 CD', 'Afid', 'Motor', 'Honda Vario', 2023, 'Daniel']
Nomor plat baru: KT 7676 HY
Nama pemilik baru: Robin
Jenis baru (Mobil/Motor): Motor
Merek baru: Honda Vario
Tahun baru: 2023
Data berhasil diperbarui!

Tekan Enter untuk melanjutkan...
```

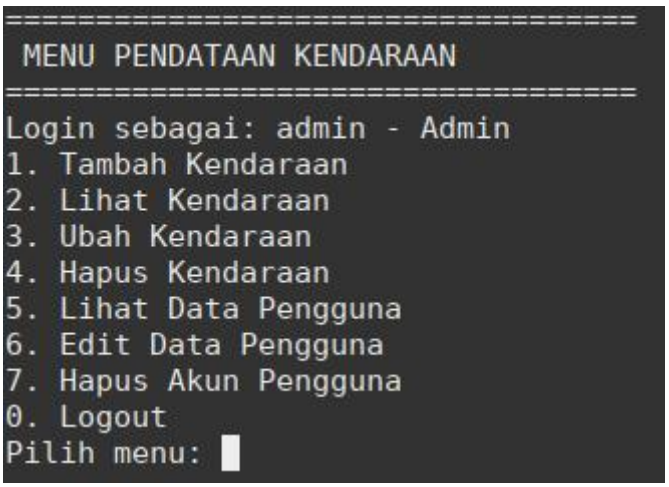
Gambar 4.5 Update



Gambar 4.6 Read



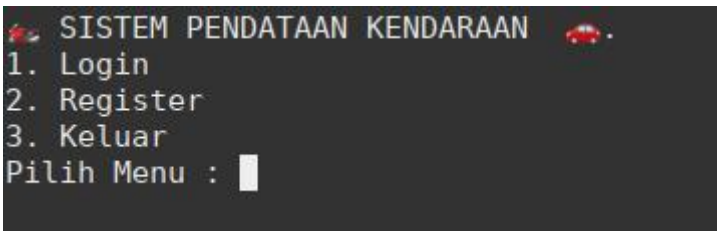
Gambar 4.7 Create



Gambar 4.8 Menu Admin



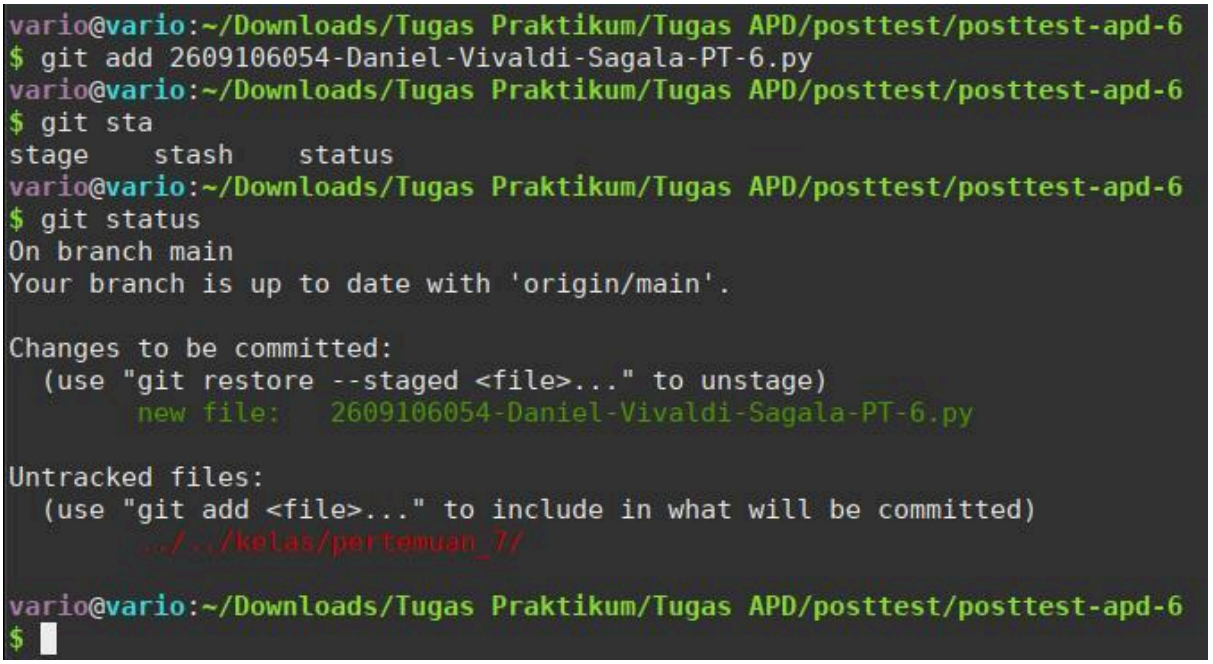
Gambar 4.9 Menu User



Gambar 4.10 Menu Utama

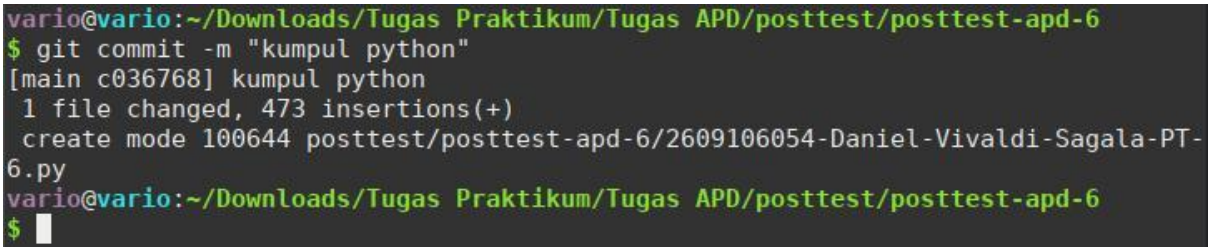
5. Langkah-langkah Git

5. 1 Git Add



Menambahkan file dengan cara "**git add nama_file**" atau jika kalian ingin menambahkan semua file kalian bisa menggunakan "**.**" Jadi "**git add .**" .lalu kalian bisa cek apakah file tersebut sudah terdaftar atau belum dengan cara mengetik "**git status**".

5.2 Git Commit



Mengetik **git commit -m "pesan yang ingin ditulis"** untuk melakukan commit atau konfirmasi perubahan yang terjadi pada repository kalian.

5.3 Git Push

```
vario@vario:~/Downloads/Tugas Praktikum/Tugas APD/posttest/posttest-apd-6
$ git push
Enter passphrase for key '/home/vario/.ssh/id_ed25519':
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 2 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 3.56 KiB | 1.78 MiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote: This repository moved. Please use the new location:
remote:   git@github.com:Ftyr3/praktikum-apd-2026.git
To github.com:Ftyr3/Praktikum_APD_B2-26.git
   625dc0d..c036768  main -> main
vario@vario:~/Downloads/Tugas Praktikum/Tugas APD/posttest/posttest-apd-6
$
```

Melakukan upload file yang tadinya hanya berada pada komputer kalian ke Github dengan cara mengetik **“git push -u origin main”**. Jika berhasil maka outputnya sama seperti pada gambar diatas.